

Chapter 6

Adversarial Attacks

While deep learning has achieved unprecedented milestones in computer vision it harbors a counterintuitive and critical vulnerability: adversarial attacks. This chapter explores how imperceptible, carefully crafted perturbations to an input image can completely subvert a neural network’s predictions. We will demystify the geometric origins of these vulnerabilities, demonstrating that they stem from the high-dimensional, linear nature of modern architectures rather than mere overfitting. By examining prominent attack vectors—ranging from the single-step Fast Gradient Sign Method (FGSM) to Black-Box optimization techniques like Boundary and One-Pixel Attacks—we uncover the exact mechanisms adversaries use to exploit these algorithmic blind spots. Finally, we transition from offense to defense, exploring adversarial training regimens, knowledge distillation, and data augmentation techniques designed to harden models against both known and unforeseen threats.

6.1 Introduction

6.1.1 Glossary and Concepts

From the perspective of explainability, we know that machine learning models (deep learning included) are prone to bias and understanding things based on small details, which may not compose what we want as prominent features. With applications from the detection of traffic signals to medical recognition, a failure in prediction or generation can have serious consequences.

Adversarial attacks primarily aim to focus on trying to learn and exploit those vulnerabilities to **confuse** the model, for example making a wrong prediction of the class of the image. However, they can also be used to enhance the **robustness** of the models training resistant models to this attacks. An example is shown in Figure 6.1, where we are trying to confuse the model to not predict the real class of the image.

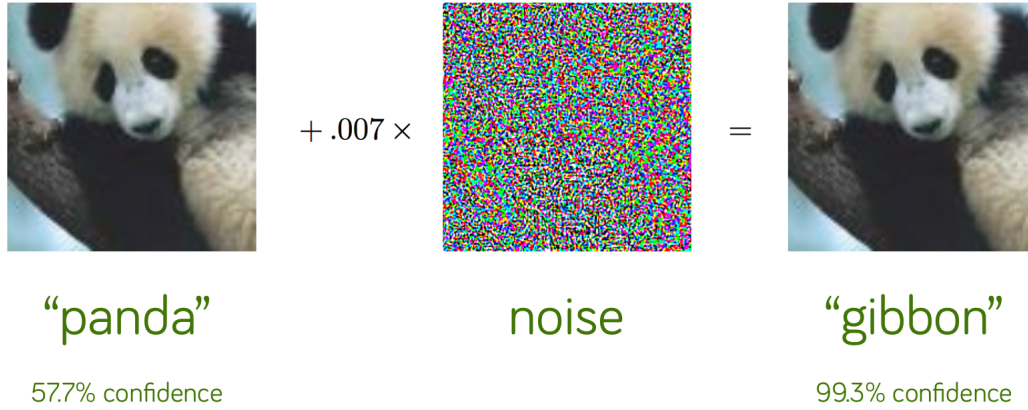


Figure 6.1: Adversarial attack to confuse the model.

Example: Fooling a Logistic Regression

Let's suppose we have a logistic regression

$$f(x) = \text{sigmoid}(w^T \cdot x + b) = \frac{1}{1 + \exp(-(w^T \cdot x + b))}.$$

let's define

$$x = (2, -1, 3, -2, 2, 2, 1, -4, 5, 1); \quad w = (-1, -1, 1, -1, 1, -1, 1, 1, -1, 1); \quad b = 0.$$

Then $f(x) \approx 0.05$. If we apply a small perturbation to x , let's say $x' = x + \varepsilon \cdot \text{sign}(w)$ with $\varepsilon = 0.5$, then $f(x') \approx 0.88$.

Before moving to specific models or techniques, let's define basic terms in Table 6.1.

Terminology	Definition
Original Example	Image present in a real dataset
Adversarial / Perturbed Example	Image that an adversary has manipulated to fool a classifier
Perturbation	Set of changes (for each pixel per channel) that are added to an original example, generating an adversarial one
Adversarial Attack	Technique used to calculate the perturbation that generates an adversarial example
Transferability	Capability of an adversarial example being transferred from a known NN model to an unknown NN model.

Table 6.1: Basic terms and definitions in natural language

To be more formal, an **adversarial example** must meet three primitives:

- The target model can make a correct prediction of the original image before a perturbation is added to it.

- The target model must make a wrong prediction of the original image after a perturbation has been added to it.
- Before and after adding perturbations to the original image, humans can make correct predictions based on their perceptions.

In **mathematical terms**, the calculation of these perturbations is an optimization problem, formally defined as:

$$\min\{\|\delta\| : f(x + \delta) = y\},$$

where f is the classifier, δ the perturbation, x the original image, and y the predicted label. The adversarial example is $x' = x + \delta$.

6.1.2 Types of Attacks

Note

This classifications vary across the literature. For example, classification according to the time rarely appears in theoretical surveys or papers. However, it appears more in cybersecurity and higher-level papers/notes.

Depending on the **goal** of the attacker, the designation is as follows:

- **Untargeted:** The attacker misleads the model to arbitrary false predictions. An example is an attack where we try the model to predict a different class than the original (just one different, not a particular one).
- **Targeted:** The attacker misleads the model to output a specific false prediction. An example is an attack where we try the model to predict a particular different class than the original (not just different, but a particular one).

Depending on the **knowledge** the attacker has access to, attack categories can be further categorised as:

- **White-box:** The attacker assumes complete knowledge of the targeted model, including its parameter values, architecture, training method, and in some cases its training data as well.
- **Black-box:** The attacker has access only to the response of the model to the inputs given.
- **Grey-box:** They assume partial model knowledge. For example, known architecture with unknown weights, or access to similar training data distributions. Some literature treats grey-box as a subset of white-box or black-box. We adopt the three-category taxonomy (white-box, grey-box, black-box) as it better reflects practical threat scenarios.

Depending on the **moment** of the attacks with respect to the life cycle of the model:

- **Training stage (poisoning attack):** The attacker injects into the training dataset adversarial samples with perturbations, or modified labels/features, intending to mislead the learning of the model by interfering with and destroying the logic of the target model.
- **Test stage (evasion attack):** The attacker produces the adversarial samples without interfering on the first stage of the training of the model, but it generates the adversarial samples with knowledge of the target model to force the model to generate wrong predictions.

According to the **perturbation budget** formulation, adversarial attacks can be categorised depending on the distance metric used to constrain the perturbation δ . This is typically formalised using L_p -norms. Some of them are shown in 6.2. Mainly, attacks operate under a perturbation budget ε , a constant that controls the amount of noise that can be added to an image via the generated perturbation, i.e., $\|\delta\|_p \leq \varepsilon$.

- L_∞ limits the largest absolute coordinate change.
- L_2 limits the Euclidean energy of the perturbation.
- L_1 limits total absolute change and can encourage concentration on fewer coordinates.
- L_0 conventionally counts changed coordinates, but it is not a norm and L_p for $0 < p < 1$ is a quasi-norm.
- In general, higher p -values emphasise minimising large errors on individual pixels and lower p -values prioritise sparse perturbations affecting fewer pixels.

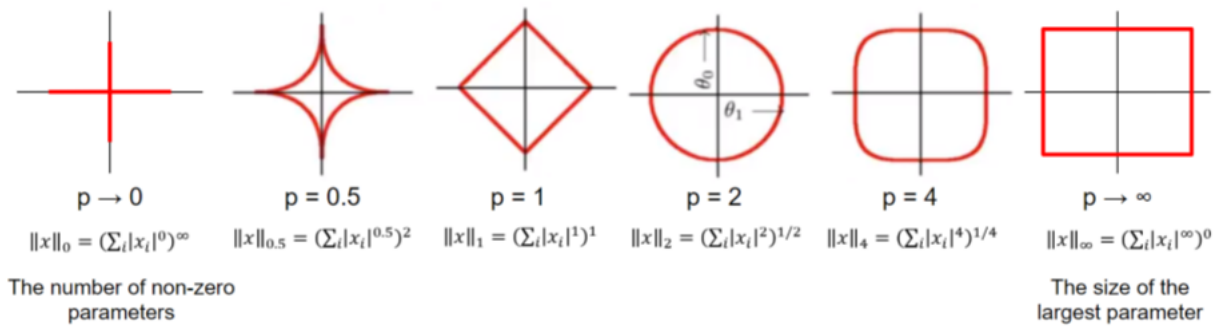


Figure 6.2: Different L_p norms.

6.1.3 Why Do Adversarial Examples Occur?

Complementary Explanations

Early explanations associated adversarial vulnerability with highly non-linear decision boundaries or overfitting. Transferability between models showed that this explanation was incomplete, but did not refute it: different models may learn correlated features and similar decision boundaries.

An influential alternative is the **linearity hypothesis**: local linear behaviour can amplify small, coordinated perturbations in high-dimensional input spaces. This view motivates FGSM, but it is not the only accepted explanation. Other perspectives involve data geometry, concentration in high dimensions, ordinary test error, and predictive but non-robust features. These should be understood as complementary explanations rather than a single settled cause.

Example: Linear analysis

Consider a linear model

$$f(\mathbf{x}) = w^T \cdot x + b,$$

where $x \in \mathbb{R}^n$ is a flattened image vector with n pixels. Applying a perturbation $\delta \in \mathbb{R}^n$ gives

$$f(x + \delta) = w^T \cdot x + b + w^T \cdot \delta = f(x) + w^T \cdot \delta.$$

The term $w^T \cdot \delta$ captures exactly how much the perturbation shifts the prediction. To understand how large this can get, we ask: given a pixel-wise budget $\|\delta\|_\infty \leq \varepsilon$ (i.e. every pixel may shift by at most ε), what choice of δ maximises $w^T \cdot \delta$?

Since $w^T \cdot \delta = \sum_i w_i \delta_i$, each term $w_i \delta_i$ is maximized independently by setting $\delta_i = \varepsilon \operatorname{sign}(w_i)$, which is admissible because $|\delta_i| = \varepsilon \leq \varepsilon$. This gives the worst-case perturbation

$$\delta^* = \varepsilon \operatorname{sign}(w),$$

and the corresponding shift in prediction is

$$w^T \cdot \delta^* = \sum_i w_i \cdot \varepsilon \operatorname{sign}(w_i) = \varepsilon \sum_i |w_i| = \varepsilon \cdot n \cdot m,$$

where $m = \frac{1}{n} \sum_i |w_i|$ is the mean absolute weight.

The general result is

$$\max_{\|\delta\|_\infty \leq \varepsilon} w^T \delta = \varepsilon \|w\|_1.$$

Writing this as $\varepsilon n m$ shows linear growth with n only under the additional assumption that the mean absolute weight m remains approximately constant as the input dimension grows. The main insight is that many small, deliberately aligned changes can accumulate into a large output shift.

Random and Adversarial Perturbations

High dimensionality does not mean that a random perturbation is likely to align with the weight vector. Random high-dimensional vectors are usually close to orthogonal. For independent symmetric random signs $\delta_i \in \{-\varepsilon, +\varepsilon\}$,

$$\mathbb{E}[w^T \delta] = 0, \quad \text{Var}(w^T \delta) = \varepsilon^2 \|w\|_2^2.$$

In contrast, the adversarial choice $\delta^* = \varepsilon \text{sign}(w)$ aligns every coordinate with the corresponding weight and produces the worst-case shift $\varepsilon \|w\|_1$. Adversarial perturbations are effective because they are deliberately optimized, not because a random direction is likely to align with the model.

Summary

No single explanation fully accounts for adversarial vulnerability. Local linearity and high-dimensional geometry explain why small worst-case perturbations can have a large effect, while data geometry and non-robust predictive features help explain why vulnerable directions are learned. The relative importance of these mechanisms depends on the model, data, and threat model.

6.1.4 Transferability

The property of an adversarial attack to be effective on multiple models is empirically seen. In particular, this means that we often don't need direct gradient access to a neural net we are actually attacking: we can just use another neural net to construct our adversarial example.

6.2 Adversarial Attack Algorithms

6.2.1 Fast Gradient Sign Method (FGSM)

The logic behind this attack is that if we know that small perturbations aligned with the sign of the gradient can lead to a maximum error (like in the previous example), then we can use the gradient of the loss the model was trained on ($\mathcal{L}$) directly to build the attack. Figure 6.3 shows an example.

This attack is “fast” in the sense that it employs a single-step computation; however, it is very weak against existing defences through algorithms and training. The single step is:

$$x_{\text{adv}} = x + \varepsilon \cdot \text{sign}(\nabla_x \mathcal{L}(y, f(x))),$$

where x is the original image, y the original label, and ε a parameter to control the budget of the perturbation.

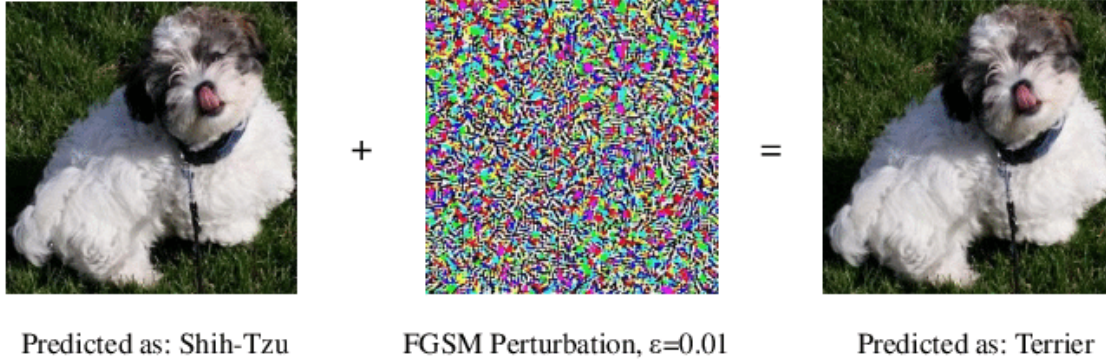


Figure 6.3: FGSM attack.

Note that this attack obeys an L_∞ budget:

$$\|x_{\text{adv}} - x\|_\infty = \|\varepsilon \cdot \text{sign}(\nabla_x \mathcal{L}(y, f(x)))\|_\infty = \|(\pm\varepsilon, \dots, \pm\varepsilon)\|_\infty = \varepsilon.$$

Warning: Value of ε

There is a **trade-off**: the bigger ε , the bigger the difference between $f(x)$ and $f(x_{\text{adv}})$, but also the bigger the difference between x and x_{adv} . Big values for ε can lead to a human detecting visually the attack.

6.2.2 Projected Gradient Descent (PGD)

FGSM takes one step in the most damaging direction of the loss, but we can increase the loss further by taking multiple small steps. The following version of Projected Gradient Descent (PGD) uses an L_∞ budget, so every update is projected onto the valid input domain and the L_∞ ball around the original image:

Algorithm 1: Projected Gradient Descent Algorithm

input : Original image x , true label y , classifier f , loss function $\mathcal{L}$, valid input domain $\mathcal{X}$, perturbation budget ε , step size α , maximum number of iterations T .

output: Adversarial example x_{adv}

- 1 Sample $u \sim \text{Uniform}([- \varepsilon, \varepsilon]^n)$;
 - 2 Initialize $x_{\text{adv}} \leftarrow \Pi_{\mathcal{X}}(x + u)$;
 - 3 Initialize $x_{\text{best}} \leftarrow x_{\text{adv}}$;
 - 4 **for** $t = 1$ **to** T **do**
 - 5 Compute gradient: $g \leftarrow \nabla_{x_{\text{adv}}} \mathcal{L}(y, f(x_{\text{adv}}))$;
 - 6 Update with step: $x_{\text{adv}} \leftarrow x_{\text{adv}} + \alpha \cdot \text{sign}(g)$;
 - 7 Project onto the feasible set: $x_{\text{adv}} \leftarrow \Pi_{\mathcal{X} \cap B_\infty(x, \varepsilon)}(x_{\text{adv}})$;
 - 8 **if** $\mathcal{L}(y, f(x_{\text{adv}})) > \mathcal{L}(y, f(x_{\text{best}}))$ **then**
 - 9 $x_{\text{best}} \leftarrow x_{\text{adv}}$;
 - 10 **return** x_{best} ;
-

Figure 6.4 shows two PGD iterations. Projection is applied after every update, although it only changes points outside the feasible set. For a stronger evaluation, the attack is usually repeated from several random initializations.

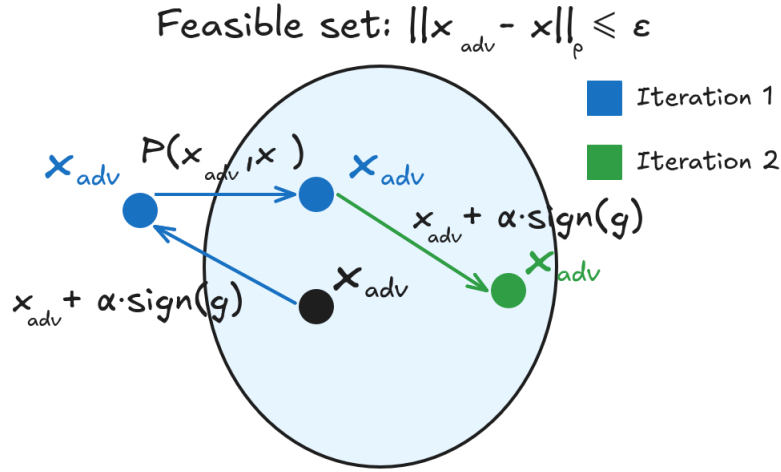


Figure 6.4: Schematic PGD projection. The rounded region represents a generic L_p ball; for the L_∞ algorithm above, the feasible region is a hypercube.

6.2.3 Carlini–Wagner (C&W)

The Carlini–Wagner attack explicitly solves an optimization problem. For the untargeted L_2 version, let $Z_j(x)$ be the logit of class j and let y be the original class. Define

$$x_{\text{adv}}(w) = \frac{1}{2} (\tanh(w) + 1)$$

and solve

$$\min_w \underbrace{\|x_{\text{adv}}(w) - x\|_2^2}_{\text{distance term}} + c \underbrace{g(x_{\text{adv}}(w))}_{\text{classification term}},$$

where

$$g(x') = \max \left(Z_y(x') - \max_{j \neq y} Z_j(x'), -\kappa \right).$$

The distance term keeps the adversarial image close to the original one. The classification term encourages another class logit to exceed the original-class logit by a margin $\kappa \geq 0$. The parameter $c > 0$ balances both terms and is commonly chosen by binary search.

The change of variable maps every finite $w \in \mathbb{R}^n$ to $x_{\text{adv}}(w) \in (0, 1)^n$, so the box constraint does not need to be handled separately. C&W is a white-box, gradient-based attack because optimizing this objective requires access to the model’s logits and gradients.

6.2.4 Boundary Attacks (BA)

This attack starts from an adversarial image and walks back toward the original image, reducing the distance while remaining adversarial. It only needs the class predicted by the

model, not logits, probabilities, or gradients. Figure 6.5 shows an orthogonal step in blue and a step toward the original image in red. The following algorithm is a simplified targeted version:

Algorithm 2: Boundary Attack Algorithm

Input : Classifier f , original image x , initial adversarial image x_0 , target class y_{target} , query budget N , orthogonal step size δ , source step size ε .

Output: Adversarial image x_{curr}

```

1  $x_{\text{curr}} \leftarrow x_0$ ;                                     //  $f(x_0) = y_{\text{target}}$ 
2 for  $k \leftarrow 1$  to  $N$  do
    /* 1. Orthogonal step */
3   Sample Gaussian noise  $\eta \sim \mathcal{N}(0, I)$  with the same shape as  $x$ 
4    $\eta_{\perp} \leftarrow \text{OrthogonalProject}(\eta, x_{\text{curr}} - x)$ 
5    $x_{\text{cand}} \leftarrow x_{\text{curr}} + \delta \cdot \text{normalize}(\eta_{\perp})$ 
    /* 2. Step toward the original image */
6    $x_{\text{cand}} \leftarrow x_{\text{cand}} - \varepsilon(x_{\text{cand}} - x)$ 
7    $x_{\text{cand}} \leftarrow \text{Clip}(x_{\text{cand}}, 0, 1)$ 
    /* 3. Accept or reject the candidate */
8   if  $f(x_{\text{cand}}) = y_{\text{target}}$  &  $\|x_{\text{cand}} - x\|_2 < \|x_{\text{curr}} - x\|_2$  then
9      $x_{\text{curr}} \leftarrow x_{\text{cand}}$ 
10     $\delta \leftarrow 1.05\delta$ ,  $\varepsilon \leftarrow 1.05\varepsilon$ ;           // increase step size
11  else
12     $\delta \leftarrow 0.95\delta$ ,  $\varepsilon \leftarrow 0.95\varepsilon$ ;       // decrease step size
13 return  $x_{\text{curr}}$ 

```

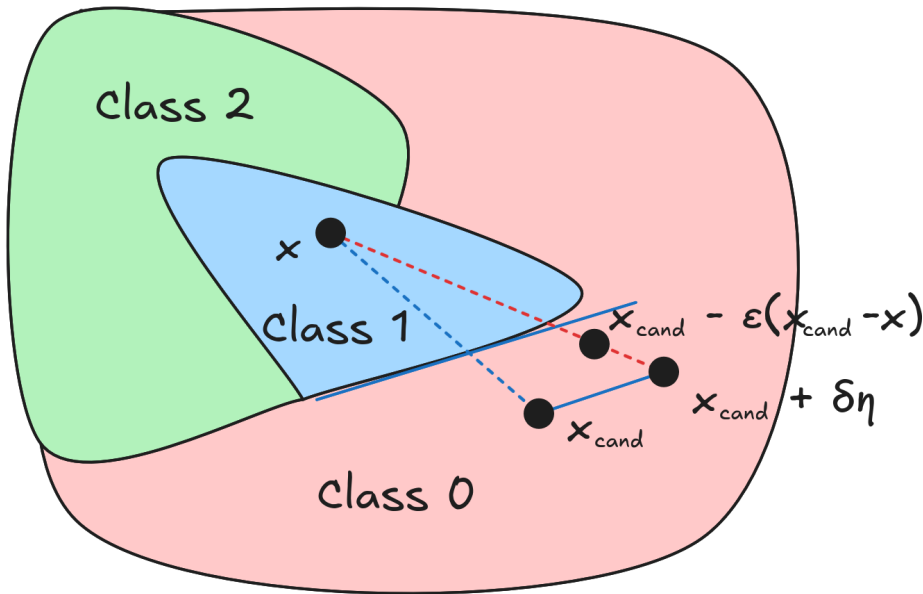


Figure 6.5: BA attack.

Note

The initial image x_0 must already be classified as the target class. It can be random noise accepted as that class or an image from the target class. During the attack iterations, it is moved closer to the original image while preserving the target prediction. This pseudocode is a simplification: the full Boundary Attack scales proposals relative to the current distance, preserves the sphere geometry more carefully, and adapts step sizes using acceptance rates over several proposals. Clipping may also alter exact orthogonality.

6.2.5 One Pixel Attack (OPA)

The idea of this attack is to generate a population $P = \{p_1, \dots, p_n\}$ where each $p_i = (x, y, r, g, b)$ represents a modification of pixel (x, y) of the image with values (r, g, b) . We will start with a random population and we will change the pixel with the best fitness value. The fitness function is $f_y(x_{\text{adv}})$, where y is the true class, and we will try to minimize it until another class is predicted.

Algorithm 3: One-Pixel Attack

input : Original image x , classifier f , true class y , population size n , maximum generations G , differential weight F .
output: Adversarial example I_{adv}

- 1 Initialize population $P = \{p_1, p_2, \dots, p_n\}$ with uniform random values;
- 2 **for** $g = 1$ **to** G **do**
- 3 **for** $i = 1$ **to** n **do**
- 4 Pick distinct random indices $r_1, r_2, r_3 \neq i$;
- 5 $v_i \leftarrow p_{r_1} + F \cdot (p_{r_2} - p_{r_3})$;
- 6 $v_i \leftarrow \text{Clip}(v_i, 0, 1)$;
- 7 $x_{\text{child}} \leftarrow \text{ChangePixel}(x, v_i)$;
- 8 **if** $\arg \max(f_y(x_{\text{child}})) \neq y$ **then**
- 9 **return** x_{child} ;
- 10 Update population with generated children;
- 11 $p_{\text{best}} \leftarrow \arg \min_{p \in P} \{f_y(\text{ChangePixel}(x, p))\}$;
- 12 $x \leftarrow \text{ChangePixel}(x, p_{\text{best}})$;
- 13 **return** x ;

There are many ways we can do the crossover. Also, we can repeat this more than once iteratively, so we can change more than just one pixel. However, the fewer pixels we change, the more similar the adversarial image will be to the original image. Finally, Figure 6.6 shows an example of this attack and Figure 6.7 the evolution of the fitness function along the training epochs.

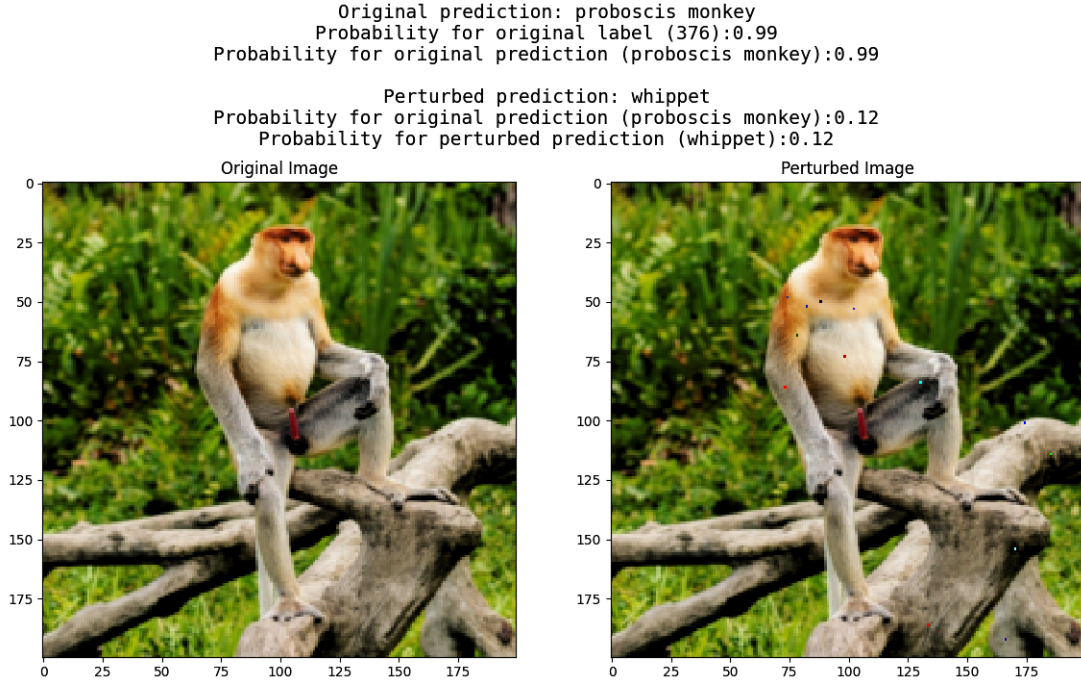


Figure 6.6: OPA attack.

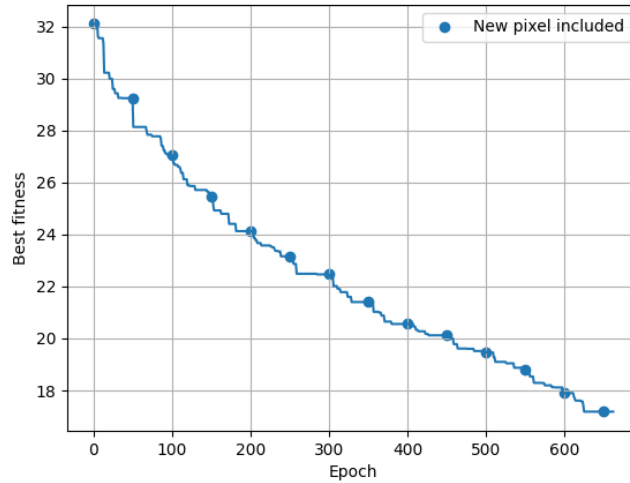


Figure 6.7: Best fitness value of all the population for each epoch during an OPA attack.

6.2.6 Comparison of the Attacks

To conclude this section, Table 6.2 compares the different attacks we have seen. Although for FGSM, CW and OPA we have seen the untargeted version, it's possible to make them targeted:

- In FGSM, we can use the gradient of the loss for the desired class instead of the initially

predicted class.

- In C&W, for a target class t , we replace the classification term with

$$g_t(x') = \max \left(\max_{j \neq t} Z_j(x') - Z_t(x'), -\kappa \right).$$

- In OPA we can use as fitness function the logits for a particular class instead of just trying to obtain a different class than the original one.

Attack	Knowledge Used	Goal
FGSM	White-Box	Untargeted
PGD	White-Box	Untargeted
C&W	White-Box	Untargeted
BA	Black-Box	Targeted
OPA	Black-Box	Untargeted

Table 6.2: Comparison of the different attacks studied.

6.3 Adversarial Training

6.3.1 Baseline Method

The most effective method for making models more robust to adversarial attacks is adversarial training. The typical procedure during the training step is:

1. Sample batch $(x_i, y_i)_{i=1}^n$.
2. Select some random indices $i_1, \dots, i_k$, $k \leq n$ and create $x_{j_{\text{adv}}} \forall j = i_1, \dots, i_k$.
3. Optimize the loss with the new batch of original samples and adversarial samples.

Warning: Number of adversarial samples during training

The more indices we select in step 2, the more robust the model will be to adversarial attacks, but it may perform worse in “normal” images.

This is the most basic example of adversarial training; however, it is not resilient enough to the most advanced attack techniques, as it can lead to memorising the perturbations, and a reduce the accuracy on clean examples.

6.3.2 Ensemble Adversarial Training

Standard adversarial training relies on generating adversarial examples exclusively against the model currently being trained. This often causes the model to overfit to its own specific decision boundary, creating a false sense of security while remaining highly vulnerable to black-box attacks transferred from other architectures.

Ensemble adversarial training mitigates this by decoupling example generation from the target model. During training, adversarial examples are crafted across a diverse ensemble of static, pre-trained models:

1. Sample batch $(x_i, y_i)_{i=1}^n$.
2. Select some random indices $i_1, \dots, i_k$, $k \leq n$.
3. For each $j = i_1, \dots, i_k$, select randomly one of the pre-trained models and generate the adversarial example $x_{j_{\text{adv}}}$ for that model.
4. Optimize the loss with the new batch of original samples and adversarial samples.

By exposing the target model to adversarial perturbations generated across multiple distinct model architectures the network learns features that genuinely generalize, achieving robust defense against transferability attacks.

6.3.3 Label Leaking

When using simple single-step attack methods during adversarial training, models can suffer from **label leaking**. Because the perturbation is constructed directly using the ground-truth label y , the resulting adversarial image can encode high-level visual artifacts of the true class. For example, in FGSM we are using the true label y to construct the adversarial sample, as $x_{\text{adv}} = x + \varepsilon \cdot \text{sign}(\nabla_x \mathcal{L}(y, f(x)))$.

Instead of learning robust visual representations, the network shortcuts training by classifying the image based on the perturbation’s leaked signal. This leads to artificially high test accuracy but catastrophic failure when exposed to iterative or truly black-box attacks at test time. Table 6.3 shows an example of this phenomenon.

		Clean	$\varepsilon = 2$	$\varepsilon = 4$	$\varepsilon = 8$	$\varepsilon = 16$
No label leaking, training and eval using “step l.l.” method	top 1	77.3%	72.8%	73.1%	73.4%	72.0%
	top 5	93.7%	91.1%	91.1%	91.0%	90.3%
With label leaking, training and eval using FGSM	top 1	76.6%	86.2%	87.6%	88.7%	87.0%
	top 5	93.2%	95.9%	96.4%	96.9%	96.4%

Table 6.3: Label leaking and no label leaking accuracies.

6.3.4 Virtual Adversarial Training

Standard adversarial training relies on ground-truth labels y to maximize cross-entropy loss, restricting its use exclusively to supervised settings. Virtual adversarial training extends adversarial regularization to semi-supervised learning by defining adversarial directions without label information.

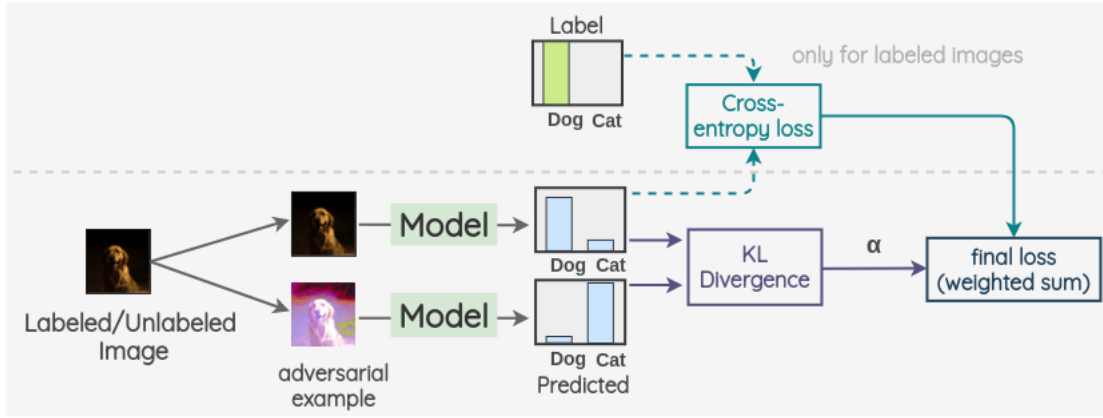


Figure 6.8: Virtual adversarial training.

Instead of maximizing loss relative to y , it measures the robustness of the model's output distribution against worst-case perturbations. The virtual adversarial perturbation r is defined as the direction that maximizes the Kullback-Leibler (KL) divergence between the model's output on the clean input x and the perturbed input $x + r$:

$$r = \arg \max_{\|r\|_2 \leq \epsilon} \{D_{\text{KL}}(f(x) \parallel f(x + r))\}.$$

Note

Because the target label is omitted during perturbation generation, this method completely avoids label leaking while enforcing output smoothness across the entire data manifold.

The objective is to minimize this divergence across both labeled and unlabeled data samples, using as loss function the **local distributional smoothness**:

$$\mathcal{L} = \mathcal{L}_{\text{CE}} + \alpha \cdot D_{\text{KL}}(f(x) \parallel f(x + r)),$$

where $\mathcal{L}_{\text{CE}}$ is the cross entropy loss for labeled data and α a parameter to control the weight of the KL part. Figure 6.8 shows the algorithm.

6.3.5 Generic Adversarial Training

Generic adversarial training can be formulated as a **min-max optimisation problem**, where the goal is to minimise the worst-case loss under adversarial perturbations. The inner maximization represents an adversary that seeks a perturbation δ within a small bounded region $\|\delta\|_p \leq \varepsilon$ to maximise the model's loss, while the outer minimization updates the model parameters θ to minimise this maximum loss. This can be expressed as

$$\min_{\theta} \mathbb{E} \left[\max_{\delta} \{ \mathcal{L}(y, f_{\theta}(x + \delta)) \} \right].$$

6.4 Robustness

6.4.1 Data Transfer and Augmentation

Adversarial robustness scales with **dataset size**. One can adversarially pretrain using adversarially distorted data for different tasks. For example, to increase CIFAR-100 adversarial robustness, first adversarially pretrain on ImageNet, a large dataset, which increases robustness by a large margin. Beyond using more data, models can also squeeze more out of existing data through data augmentation.

CutMix is a data augmentation technique that, when combined with adversarial training, yields strong performance gains. A rectangular patch is cut from a secondary image x_B and pasted directly onto a target image x_A .

Given a binary bounding-box mask $M \in \{0, 1\}^{W \times H}$, where W and H represent the image width and height, values of 1 retain the original pixels of x_A , while 0s define the rectangular region replaced by x_B . The augmented image $\tilde{x}$ and its corresponding label $\tilde{y}$ are formulated as:

$$\tilde{x} = M \odot x_A + (\mathbf{1} - M) \odot x_B; \quad \tilde{y} = \lambda y_A + (1 - \lambda) y_B.$$

To construct the sample, the primary image ratio $\lambda \in (0, 1)$ is drawn from a Beta distribution $\text{Beta}(\alpha, \alpha)$. The bounding-box dimensions are then chosen such that the cut patch covers an area of exactly $(1 - \lambda) \cdot (W \times H)$. Consequently, the target label weight $\lambda = 1 - \frac{\text{area}(M)}{W \times H}$ perfectly mirrors the spatial proportion of x_A remaining in the final image.

By forcing the network to classify partial visual cues across localized spatial regions, CutMix prevents over-reliance on single dominant features, boosting both standard classification accuracy and robustness against localized Occlusions and domain shifts.

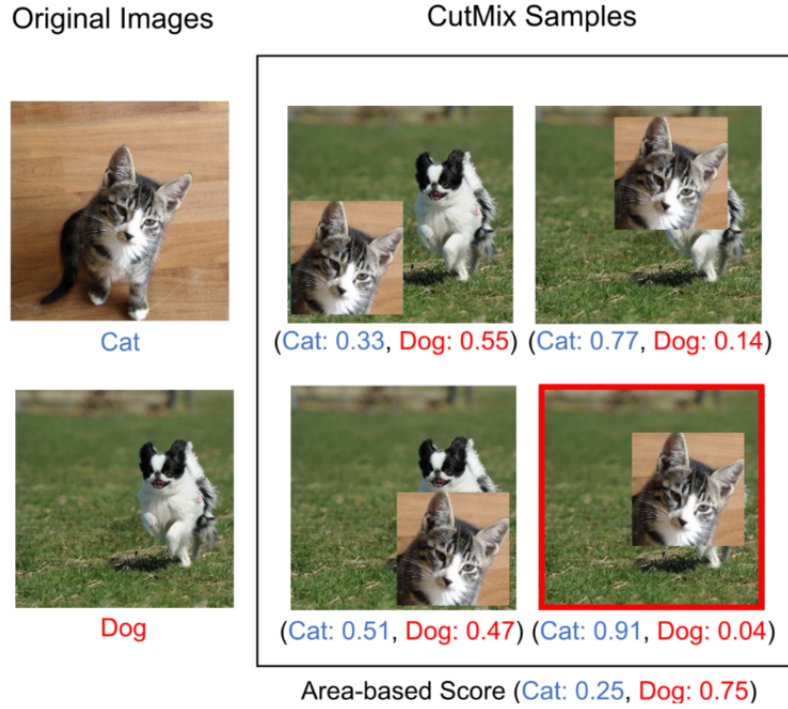


Figure 6.9: CutMix examples.

6.4.2 Smooth Activations

Sharp activation functions, such as ReLUs, make adversarial training less effective. By improving gradient quality for either the adversarial attacker or the network optimiser, smooth activations such as GELU improve adversarial training (see Figure 6.10 to compare visually these activations). The downside of the problem is that it makes training more difficult.

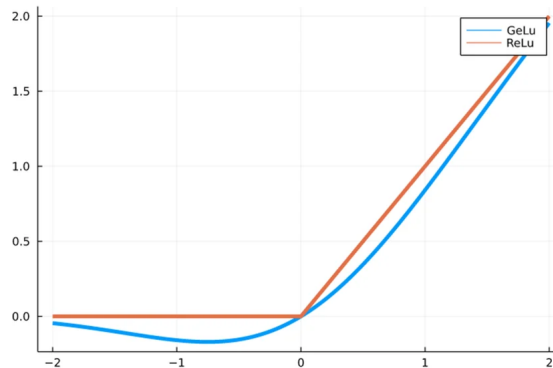


Figure 6.10: GELU vs ReLU.

6.4.3 Defensive Distillations

Knowledge distillation has emerged as a key alternative (or complement) to standard adversarial training. In this framework, a robust **teacher** model is trained first, and its learned representations are subsequently transferred to a **student** network. The student model is optimized using a two-stage or ensembled pipeline, learning from the continuous probability distributions (soft labels) generated by the teacher rather than the conventional ground-truth one-hot vectors (hard labels). Figure 6.11 shows the whole process.

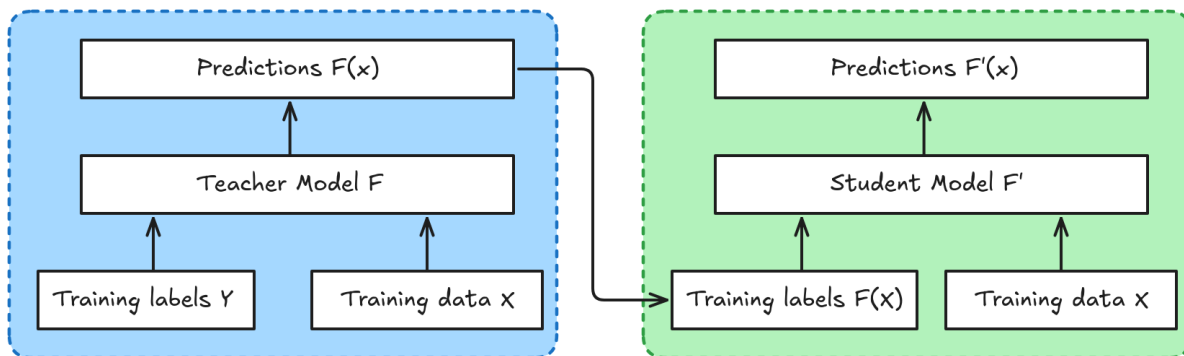


Figure 6.11: Knowledge distillation algorithm.

This approach allows a simpler or compressed student model to inherit adversarial robustness without requiring direct, compute-heavy adversarial training on the student itself—even when the student uses a completely different architecture from the teacher.

However, this method is not infallible. Distilled models remain susceptible to optimization-based attacks such as Carlini & Wagner (C&W), as soft labels can sometimes induce gradient masking rather than genuine robustness. Furthermore, the pipeline adds training complexity and high computational overhead, as both teacher and student models must be trained sequentially.

6.4.4 Unforeseen Adversaries

A major limitation of adversarial training is that models typically fail to generalize to perturbation types not encountered during training. For example, a model trained specifically to defend against bounded L_∞ pixel perturbations can remain vulnerable to unconstrained, real-world threats.

In practical deployment, attackers may exploit novel or unconstrained attack vectors whose threat models cannot be defined *a priori*. Consequently, good practice in safety-critical applications relies on multi-attack adversarial training—exposing the model to a broad mixture of distinct attack topologies during training to minimize the occurrence of out-of-distribution threats at test time.

6.5 Exercises

Exercise 1: Linear Models

Explain the linearity hypothesis for adversarial examples and compare it with one complementary explanation. For $f(x) = w^T x + b$, derive the largest output change under an L_∞ budget and state the assumption needed for this change to grow linearly with the input dimension n .

Exercise 2: Knowledge of the Attacks

Compare and contrast white-box, grey-box, and black-box threat models. Explain the phenomenon of adversarial transferability and why it allows an attacker to execute effective black-box attacks against proprietary models.

Exercise 3: FGSM and PGD

Compare FGSM and PGD from an optimization perspective. Why is PGD considered a much stronger attack, and what is the exact mathematical role of the projection operator $\mathcal{P}_\varepsilon$ during its execution?

Exercise 4: CW attack

In the Carlini-Wagner (CW) attack, explain why the change of variable $x_{\text{adv}} = \frac{1}{2}(\tanh(w) + 1)$ is applied instead of optimizing the pixel values directly.

Exercise 5: Implementation of FGSM

Write a PyTorch function that takes a classification model, clean images scaled to $[0, 1]$, their labels, and a budget ε . Implement untargeted FGSM and return the adversarial images clamped to $[0, 1]$.

Exercise 6: Implementation of PGD

Write a Python class for the L_∞ PGD attack with parameters T , α , and ε . Use a random start, run all T iterations, and after every update project onto the ε -ball around the original image and clamp the result to $[0, 1]$.

Exercise 7: Implementation of Boundary Attack

Implement the simplified targeted Boundary Attack described in this chapter. Start from an image already classified as the target class, use orthogonal Gaussian proposals, and move candidates toward the original source image while preserving the target prediction. Include a query budget.

Exercise 8: CutMix

Write a PyTorch function that implements CutMix. Sample $\lambda \sim \text{Beta}(\alpha, \alpha)$, use it to choose a patch, and paste that patch from x_B onto x_A . Recompute λ_{real} from the actual clipped box and use it to mix the labels.

Exercise 9: Adversarial Training

Write a PyTorch training loop for PGD adversarial training. For each mini-batch, generate and detach adversarial examples without updating the model parameters, and then update the model using the loss on those examples.

Exercise 10: Local Distributional Smoothness

Implement the Virtual Adversarial Training loss in PyTorch without using ground-truth labels. Treat the clean prediction as a fixed target, initialize a small random L_2 -normalized direction, and use a finite-difference power iteration to estimate the adversarial perturbation.

Exercise 11: Robustness Evaluation

Write an evaluation script that compares a standard model with an adversarially trained model. Report clean and adversarial Top-1 accuracy over the test set using multi-step L_∞ PGD with a random start. Choose a range of budgets appropriate to the image scale, state the attack settings, and plot both robustness curves.